

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment**

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	Rashmi Naurene	Reg. No.:	192521357
Date:	5.08.2026	Duration:	60 minutes

Code of Conduct

I _____ Rashmi Naurene (192521357) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Rashmi Naurene _____

Annexure A – Architecture Design Assignment**Instructions to Students:**

Each student will be assigned an individual software project problem statement. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your assigned problem statement, perform an Architecture Design Analysis and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- ☐ Study the given problem statement and identify the system objectives.
- ☐ Analyze the functional and non-functional requirements that influence the software architecture.
- ☐ Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- ☐ Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- ☐ Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- ☐ Design a clear architecture diagram illustrating the overall structure of the proposed system.
- ☐ Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- ☐ Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- ☐ Describe the purpose and responsibilities of each architectural component.
- ☐ Explain how the components communicate and exchange data.
- ☐ Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- ☐ Scalability
- ☐ Security
- ☐ Performance
- ☐ Reliability
- ☐ Maintainability
- ☐ Availability

6. Justify Architectural Decisions

- ☐ Explain the rationale behind your architectural choices.
- ☐ Discuss the trade-offs considered while selecting the architecture.
- ☐ Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Problem Statement Domain: Employee Skill-Gap Analysis and Training Recommendation System — a platform that evaluates employee competencies against role requirements, identifies deficiencies through data analytics, and recommends personalized training, with manager dashboards and automated learning notifications.

1 · System Requirements Analysis

1.1 System Objectives

The architecture must support eight objectives drawn directly from the problem statement: evaluating employee competencies, identifying skill gaps against role requirements, recommending personalized training, tracking learning progress and certifications, providing HR dashboards and reports, supporting competency-based career development, improving productivity through continuous learning, and enabling data-driven training decisions. These objectives span three distinct workloads — transactional (profile updates), analytical (gap computation), and reporting (dashboards) — which is the single most important factor shaping the architecture choice made in Section 2.

1.2 Functional Requirements Influencing Architecture

- Profile intake from multiple sources (self-reported, HRMS import, manager endorsement) — implies an integration layer independent of the core analysis logic.
- Automated, potentially batch-heavy comparison of profiles against competency frameworks — implies a computationally isolated service that can scale independently of user-facing traffic.
- Ranked training recommendations generated from gap output — implies a downstream consumer that reacts to analysis results, naturally suited to asynchronous/event-based triggering.
- Manager and HR dashboards querying aggregated data — implies a read-optimized reporting path separate from the transactional write path.
- Automated notifications tied to learning-activity deadlines — implies a scheduled, decoupled delivery mechanism.

1.3 Quality Attributes Driving the Architecture

Quality Attribute	Why It Matters for This System
Scalability	Gap-analysis and reporting workloads grow with headcount (1,000 → 50,000 employees) and must scale independently of the lighter-weight profile/login traffic.
Maintainability	Competency frameworks, course catalogues, and recommendation logic will change frequently as HR policy evolves; these must be updatable without redeploying the entire platform.
Reliability	A failure in the notification or reporting path must not block an employee from logging in or submitting a certification.
Availability	Dashboards and the employee portal are consulted during business hours across departments and must remain available even during batch analysis runs.
Performance	The nightly gap-analysis batch must complete within a fixed operational window; recommendation queries must return within interactive response times.
Security	Competency, performance, and certification data is sensitive HR data and must be protected with role-based access control and encryption in transit and at rest.

2 · Selected Software Architecture

2.1 Architecture Style: Layered (Object-Oriented) Architecture

The proposed system adopts a Layered, object-oriented architecture, organized into three conceptual layers that map directly onto the classes in the design: an Actor/Controller layer (Manager, HRAdmin) representing the human roles that drive the system; a Service layer (SkillGapAnalysis, RecommendationEngine, Dashboard, Notification) that encapsulates the system's core behaviour; and a Domain/Entity layer (Employee, JobRole, Skill, Assessment, Certification, TrainingProgram, ProgressTracker) that holds the persistent data those services operate on.

2.2 Choosing the UML Diagram: Class Diagram

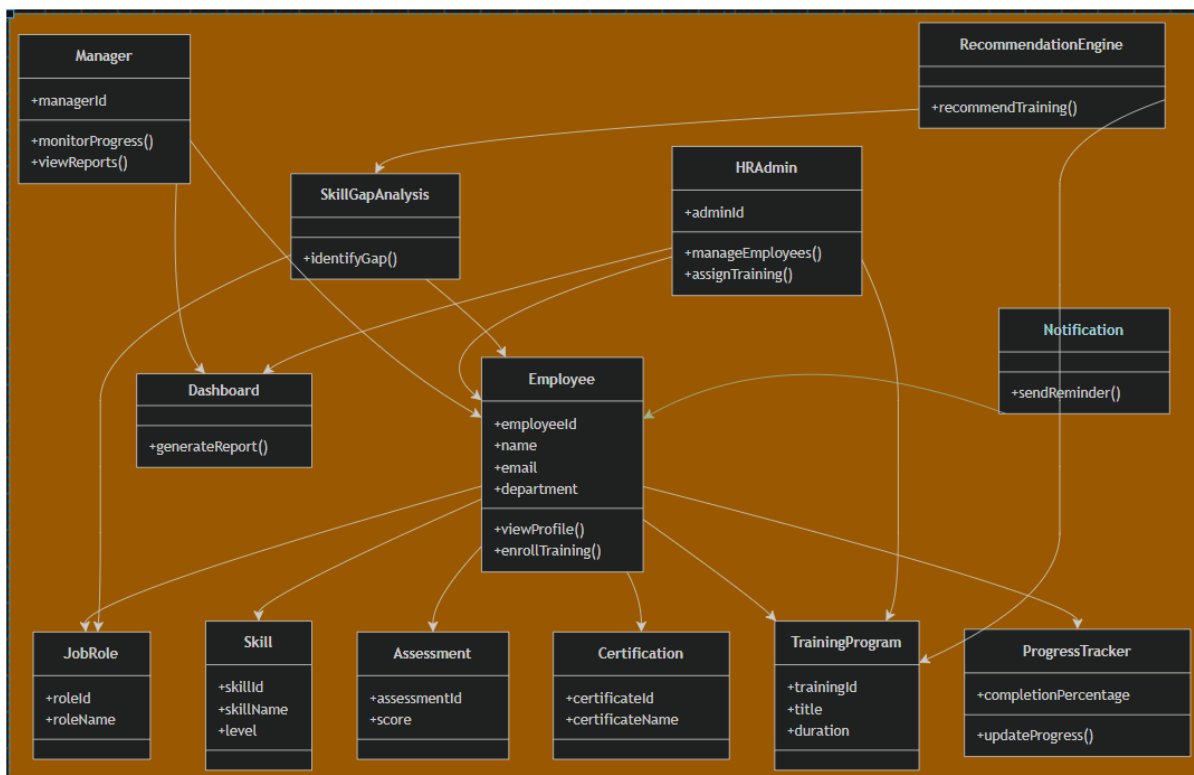
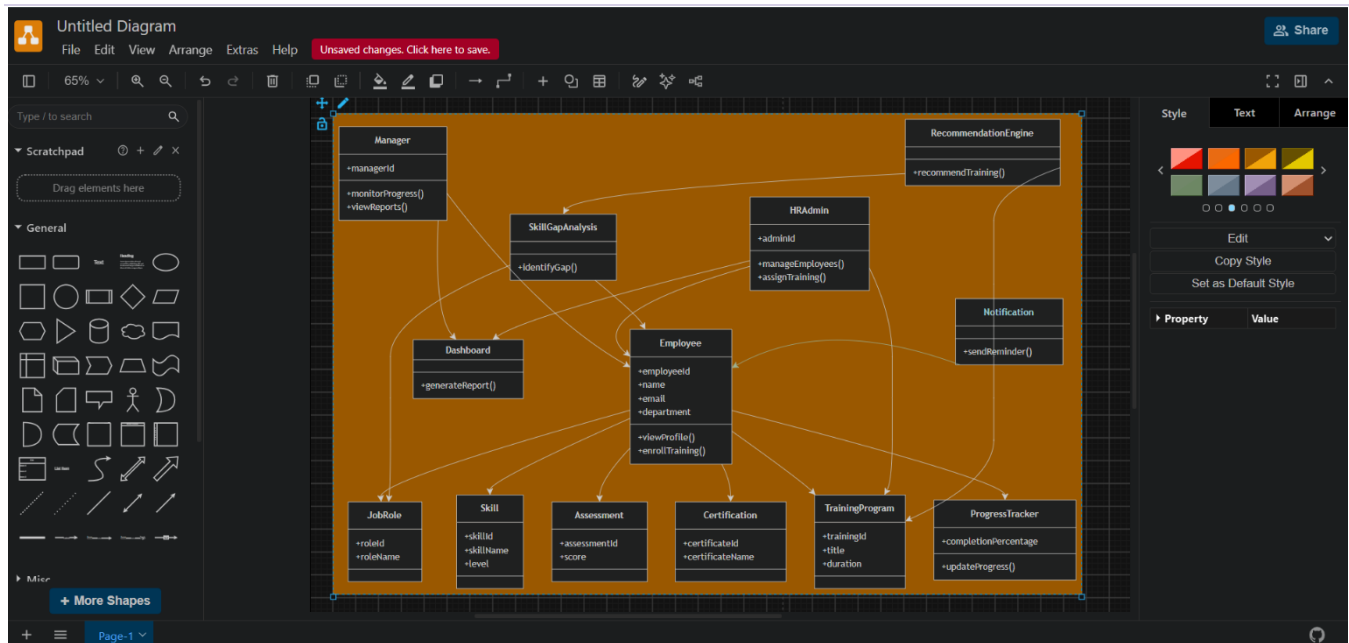
A UML Class Diagram was used to represent the architecture because the design is expressed at the object level: every class shown carries the attributes it owns and the operations it exposes, and the associations between classes are exactly the collaboration paths the system relies on — for example, SkillGapAnalysis is associated with both Employee and JobRole because identifying a gap requires comparing one against the other. A Component Diagram would be appropriate once these classes are grouped into deployable services, and a Deployment Diagram once physical hosting is decided, but at this stage of the design a Class Diagram gives the most direct, implementation-ready picture of the system's structure — and is the diagram actually authored for this submission using a UML modelling tool.

2.3 Justification for the Architecture Style

- Direct traceability to code: each class in the diagram becomes a class in the implementation, so the architecture and the eventual codebase stay in sync with minimal translation.
- Separation of concerns: Actor classes only initiate actions, Service classes only compute, and Entity classes only hold data — no class mixes these responsibilities, which keeps each one easy to reason about in isolation.
- Appropriate for current scale: the domain is centred on one primary entity (Employee) with a handful of related records; a full distributed/microservices split would add operational overhead without a matching scalability need yet.
- Natural fit for an academic/early-stage project: a layered class design is straightforward for a small team to build, test, and extend, while still leaving a clear path to decompose the Service layer into independent services later (Section 6.3).
- Matches the actual workflow: the real process — a Manager/HRAdmin acting on an Employee, a gap being identified, a recommendation being generated, a reminder being sent — is a chain of object collaborations, which is exactly what the class associations in Section 3 capture.

3 · Software Architecture Diagram

The diagram below is the UML Class Diagram modelling the system's structure: the Actor classes (Manager, HRAdmin) at the top drive the Service classes (SkillGapAnalysis, RecommendationEngine, Dashboard, Notification) in the middle, which in turn read and update the Entity classes (Employee and its related records) along the bottom. Each class shows its key attributes and operations, and the connecting lines show which classes collaborate directly.



4 · Components and Their Interactions

4.1 Component (Class) Responsibilities

Class	Layer	Responsibility
Manager	Actor	Monitors employee progress and views reports; references JobRole and Employee directly, and triggers Dashboard for aggregated views.

HRAdmin	Actor	Manages employee records and assigns training; the only class with authority to add/modify Employee accounts, and also triggers Dashboard.
SkillGapAnalysis	Service	Compares an Employee's record against a JobRole's requirements and identifies the gap; its output is what the RecommendationEngine acts on.
RecommendationEngine	Service	Generates a training recommendation from an identified gap, links the Employee to a TrainingProgram, and triggers the Notification class.
Dashboard	Service	Produces reports consumed by Manager and HRAdmin; the single reporting surface for both actor roles.
Notification	Service	Sends reminders to keep an Employee's assigned training moving; triggered by the RecommendationEngine once a suggestion is ready.
Employee	Entity	The central domain object; holds identity/profile data and is the hub that Skill, Assessment, Certification, TrainingProgram, and ProgressTracker all relate back to.
JobRole	Entity	Defines the target competency profile for a role; referenced by both Manager and SkillGapAnalysis.
Skill / Assessment / Certification	Entity	Hold the individual pieces of evidence (competency, score, credential) that make up an Employee's profile.
TrainingProgram	Entity	Represents a course an Employee can enrol in; populated both by direct enrolment and by the RecommendationEngine's suggestions.
ProgressTracker	Entity	Tracks completion percentage for a training item and feeds that status back onto the Employee record.

4.2 System Workflow

The class associations trace a single coherent workflow. (1) Manager and HRAdmin both act directly on Employee — HRAdmin manages the record and assigns training, Manager monitors progress — and both also consult JobRole where role requirements are relevant. (2) SkillGapAnalysis reads an Employee's record together with the corresponding JobRole and identifies the gap between them via `identifyGap()`. (3) RecommendationEngine works from that gap: it calls `recommendTraining()` to match the Employee to a suitable TrainingProgram, and immediately triggers `Notification.sendReminder()` so the employee is prompted to act. (4) The Employee enrolls with `enrollTraining()`; as the course progresses, ProgressTracker measures `completionPercentage` and calls `updateProgress()`, which is reflected back onto the Employee's own record. (5) At any point, Manager and HRAdmin can call `Dashboard.generateReport()` to see the aggregated picture — gaps identified, recommendations issued, and training completed — without needing to query individual Employee records directly.

5 · Quality Attribute Evaluation

Scalability

New capabilities attach cleanly at the layer they belong to: a new evidence type would be added as another Entity class alongside Skill, Assessment, and Certification without touching the Service layer, while a new computation

would be added as a new Service class without altering the Entity classes it reads. This class-level separation is what will let the Service layer later be split into independently scalable units, as described in Section 6.3.

Security

Access is naturally scoped by class: only HRAdmin exposes `manageEmployees()` and `assignTraining()`, only Manager exposes `monitorProgress()` and `viewReports()`, and Employee exposes only self-service operations (`viewProfile()`, `enrollTraining()`). Enforcing that each actor class may only invoke its own operations gives the system role-based access control at the design level, before any framework-level permission system is even introduced.

Performance

SkillGapAnalysis and RecommendationEngine are kept as dedicated Service classes rather than being folded into Employee itself, so the comparison and ranking logic can be optimized or cached independently of simple profile reads such as `viewProfile()`. `Dashboard.generateReport()` aggregates across entities rather than the Manager/HRAdmin querying each Employee record individually, keeping report generation a single, optimizable operation.

Reliability

Notification is a separate class from RecommendationEngine, so a failure in `sendReminder()` does not roll back or block the recommendation itself — the recommendation is already associated with the Employee and TrainingProgram before the reminder is attempted, so delivery issues degrade gracefully rather than losing work.

Maintainability

Each class has a small, well-defined set of attributes and operations — ProgressTracker, for example, knows only `completionPercentage` and `updateProgress()`, nothing about the Employee's personal data. This high cohesion means a change to how progress is calculated cannot accidentally affect how a profile is displayed, which keeps maintenance changes localized and low-risk.

Availability

Because Dashboard and Notification are separate classes from the core Employee/TrainingProgram data, an outage in reporting or reminders does not prevent the core actions — `viewProfile()`, `enrollTraining()`, `identifyGap()` — from continuing to function.

6 · Architectural Decisions & Trade-offs

6.1 Rationale

The core decision was to model the system as three class layers — Actor, Service, Entity — rather than jumping straight to an infrastructure-level design. This was chosen because, at this stage, the problem is fundamentally about relationships between people (Manager, HRAdmin, Employee) and structured competency data (Skill, Assessment, Certification, TrainingProgram); a class-based domain model captures those relationships directly and gives a design that translates almost one-to-one into code.

6.2 Trade-offs Considered

- Class-level layering vs. microservice decomposition: splitting each Service class into its own deployable microservice was considered but rejected for now — Employee is central to nearly every class, and separating these into different services this early would introduce distributed-transaction complexity without a scalability problem to justify it yet.
- Direct associations vs. interface-based decoupling: the diagram connects classes directly (e.g., Employee is associated with five different entity classes) rather than routing every interaction through an abstraction layer. This keeps the model simple and easy to read, at the cost of tighter coupling — judged an acceptable trade for the project's current size.
- A single central Employee entity vs. distributing identity data: keeping Employee as the one class that Skill, Assessment, Certification, TrainingProgram, and ProgressTracker all relate back to makes a single profile view trivial to build, but it does create a hotspot class that any future refactor will need to treat carefully.

6.3 Support for Future Enhancements

The Service layer (SkillGapAnalysis, RecommendationEngine, Dashboard, Notification) is deliberately kept separate from the Entity layer specifically so it can be lifted out later: once usage grows, each Service class can be promoted to its own deployable component sitting behind an API Gateway, communicating asynchronously through a message broker — the same integration pattern outlined in an earlier architecture draft for this system — without needing to redesign the underlying Employee/JobRole/Skill domain model captured here. This gives the project a clear, low-risk growth path from the current class-based design to a distributed one when the scale actually demands it.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis..	Requirement analysis is incomplete, inaccurate, or lacks clarity..
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

